# SENG 201
# Software Engineering Foundations

[Week 4, Monday]

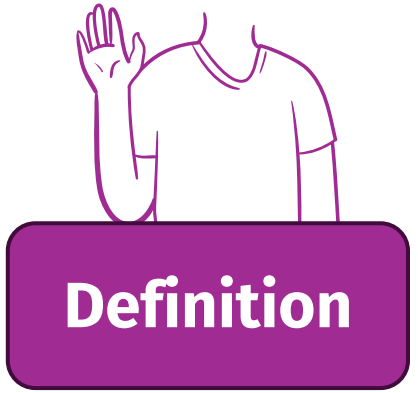


# This week in SENG201...

Types and Inheritance


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

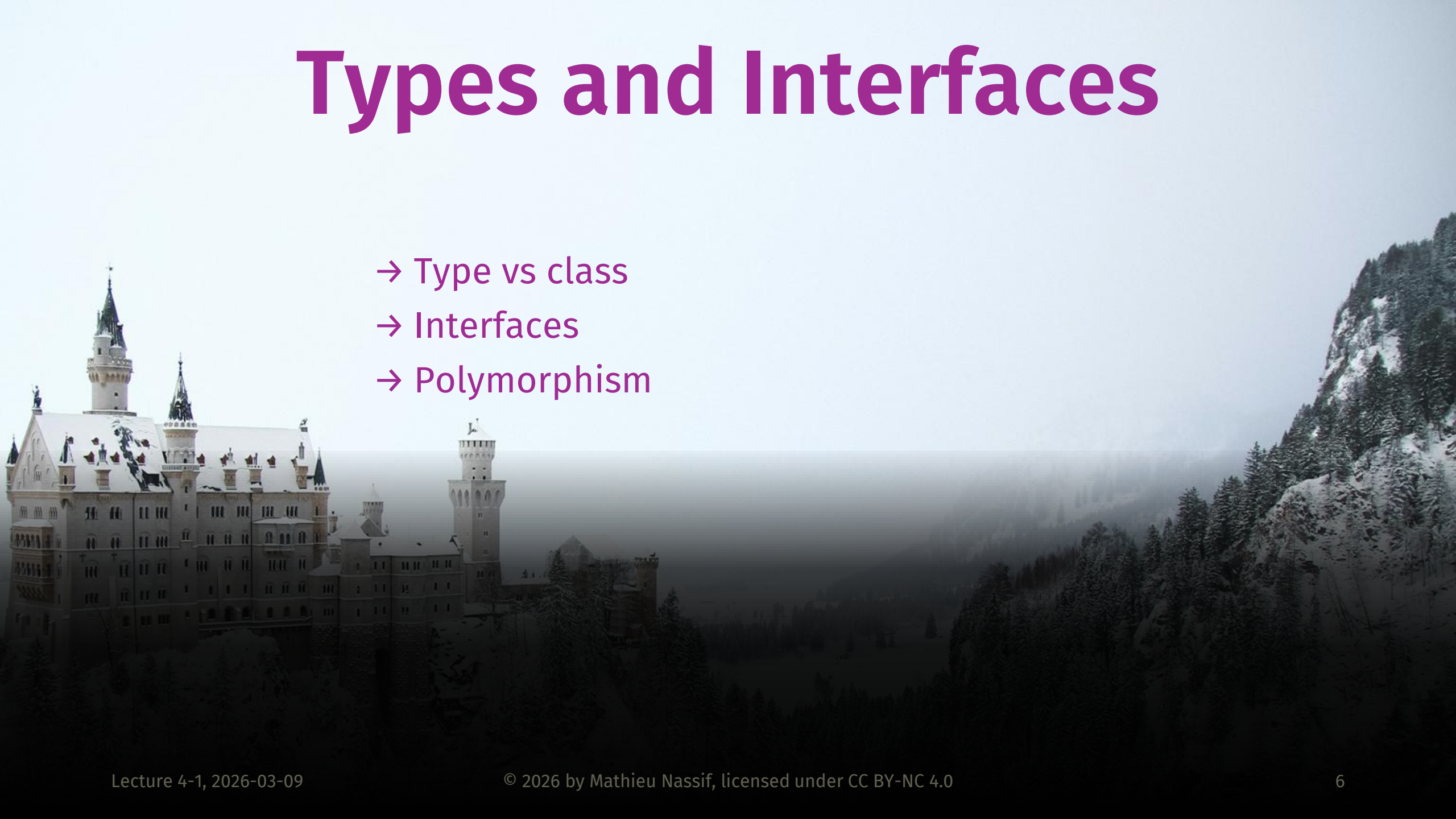
# Remember these terms?




Class Object Data type Interface


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Types and Interfaces



→ Type vs class

→ Interfaces

→ Polymorphism


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Different types of Notebook content

```java
public class Link {

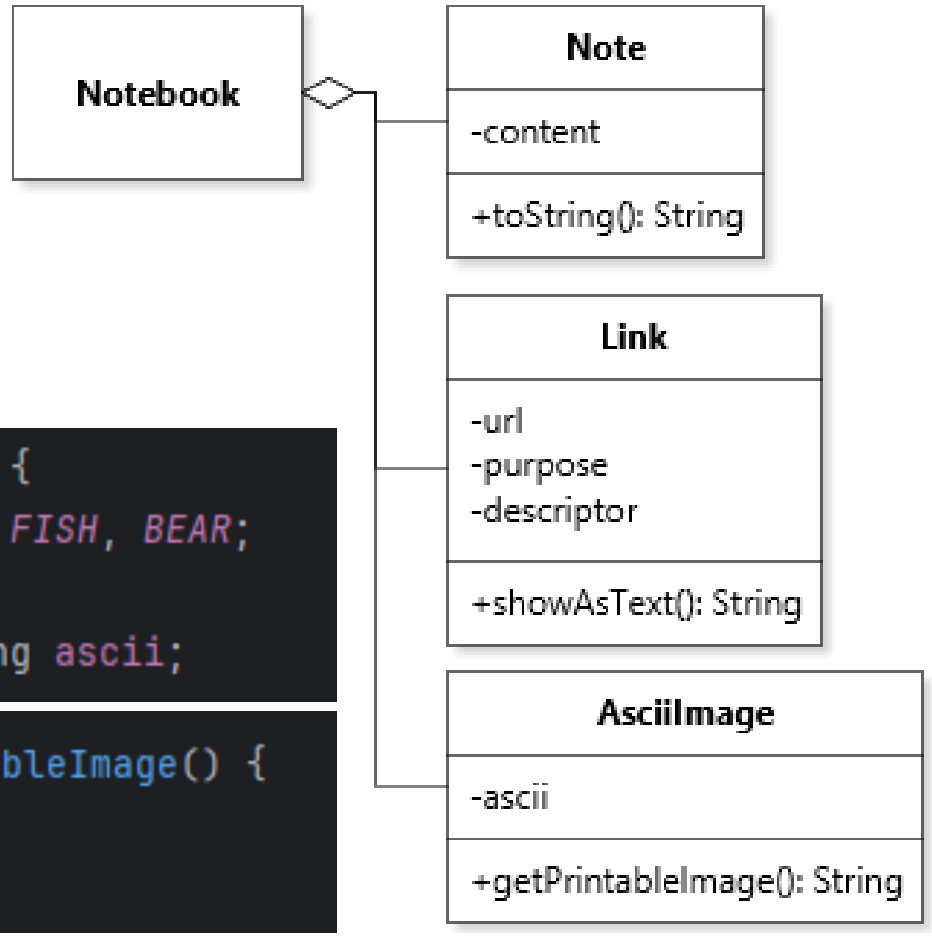
    private final String url;
    private final String purpose;
    private String descriptor;
```

```java
public String showAsText() {
    String linkText;
    if (descriptor != null) {
        linkText = "[" + purpose + "] "
                + descriptor + "<" + url + ">";
    }
    else {
        linkText = "[" + purpose + "] " + url;
    }
    return linkText;
}
```

```java
public enum AsciiImage {
    CAT, DOG, DOLPHIN, FISH, BEAR;

    private final String ascii;
```

```java
public String getPrintableImage() {
    return ascii;
}
```





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Different types of Notebook content

```java
private final List<ContentType> allContent = new ArrayList<>();
private final List<Note> notes = new ArrayList<>();
private final List<AsciiImage> images = new ArrayList<>();
private final List<Link> links = new ArrayList<>();

public void writeNote(Note note) {
    allContent.add(ContentType.NOTE);
    notes.add(note);
}

public void addImage(AsciiImage image) {
    allContent.add(ContentType.IMAGE);
    images.add(image);
}

public void addLink(Link link) {
    allContent.add(ContentType.LINK);
    links.add(link);
}
```

```java
public String readAllNotes() {
    String fullContent = "Notebook for " + createCourseString();
    fullContent += "\n--------";
    int noteIndex = 0;
    int imageIndex = 0;
    int linkIndex = 0;
    for (ContentType content : this.allContent) {
        switch (content) {
            case NOTE:
                fullContent += "\n" + notes.get(noteIndex).toString();
                noteIndex++;
                break;
            case IMAGE:
                fullContent += "\n" + images.get(imageIndex).getPrintableImage();
                imageIndex++;
                break;
            case LINK:
                fullContent += "\n" + links.get(linkIndex).showAsText();
                linkIndex++;
                break;
            default:
                fullContent += "\n    ERROR: unrecognized type";
        }
    }
    return fullContent;
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Creating a new class does 2 things

## Blueprint for creating objects

- What fields the objects should have

```
private final String url;
private final String purpose;
private String descriptor;
```

- How to construct the objects

```
public Link(String url, String purpose) {
    this.url = url;
    this.purpose = purpose;
```

- How to implement each method

```
public String showAsText() {
    String linkText;
    if (descriptor != null) {
        linkText = "[" + purpose + "] "
```

## New type

- Data type for variables

```
Link link;
List<Link> listOfLinks = new ArrayList<>();
```

- Methods (behaviours) of the variable

```
link = new Link("https://www.asciiart.eu", "Essential");
link.setDescriptor("ASCII art snippets");
String text = link.showAsText();
```

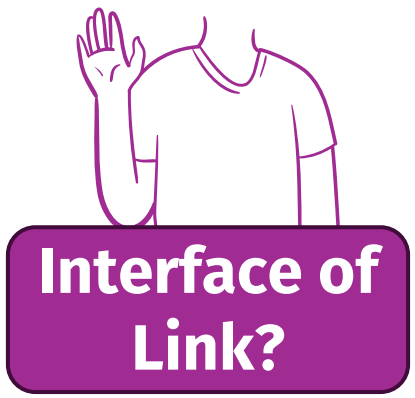

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

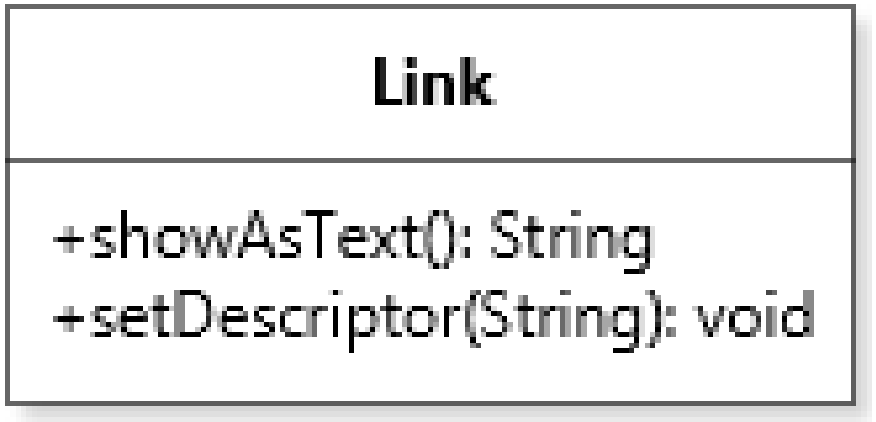
# ~~Classes~~ define abstractions
# Types

| | | |
|---|---|---|
| `Notebook` | → | A collection of notes about a course |
| `Link` | → | A hyperlink to an online reference |
| `String` | → | A sequence of characters |
| `char` | → | A single character (implemented as an integer) |


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Types define abstractions + interfaces




**Interface of a class:**

List of members that other classes have access to

- Access modifiers set the interface
- We can call `link.showAsText()` not because it's part of the object, but because it's part of the interface


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Can we separate interfaces from classes?


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Define interfaces without classes

Blueprint for creating objects

- What fields the objects should have

```
private final String url;
private final String purpose;
private String descriptor;
```

- How to construct the objects

```
public Link(String url, String purpose) {
    this.url = url;
    this.purpose = purpose;
```

- How to implement each method

```
public String showAsText() {
    String linkText;
    if (descriptor != null) {
        linkText = "[" + purpose + "] "
```

New type (interface)

- Data type for variables

```
Link link;
List<Link> listOfLinks = new ArrayList<>();
```

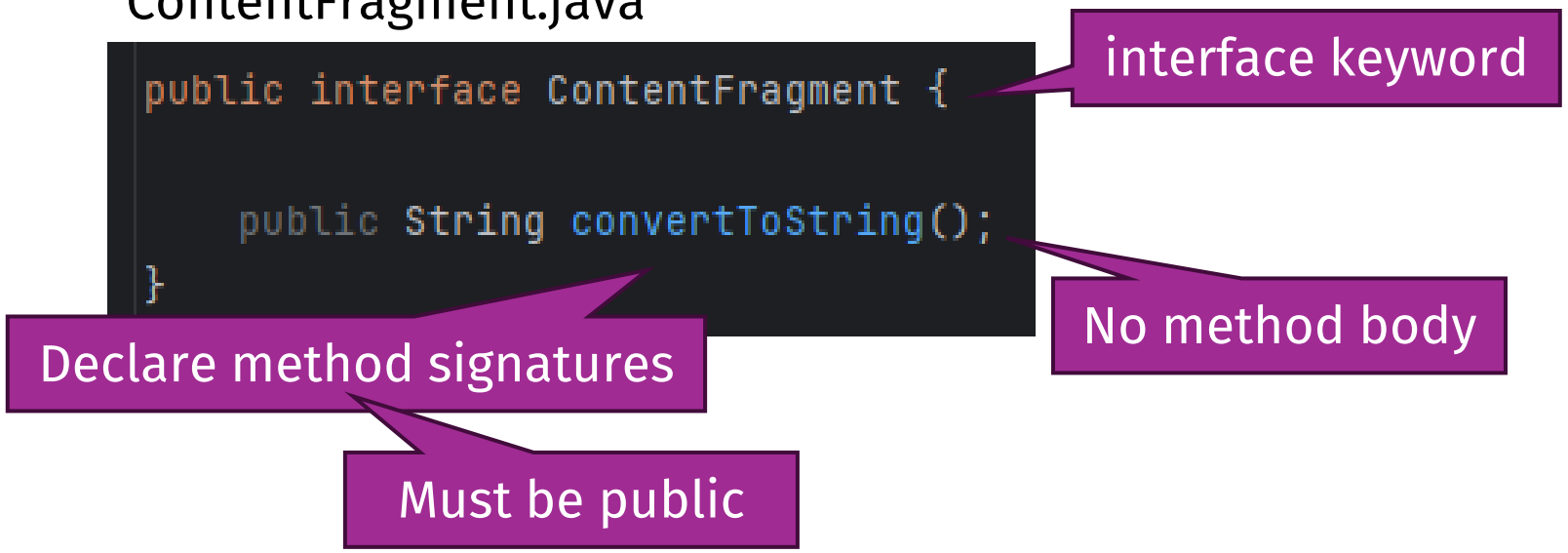
- Methods (behaviours) of the variable

```
link = new Link("https://www.asciiart.eu", "Essential");
link.setDescriptor("ASCII art snippets");
String text = link.showAsText();
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Define interfaces without classes

ContentFragment.java





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Define interfaces without classes

ContentFragment.java

```java
public interface ContentFragment {

    public String convertToString();
}
```

Link.java

```java
public class Link implements ContentFragment {
```

```java
@Override
public String convertToString() {
    String linkText;
    if (descriptor != null) {
        linkText = "[" + purpose + "] "
```

Notebook.java

```java
case LINK:
    fullContent += "\n" + links.get(linkIndex).convertToString();
    linkIndex++;
    break;
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Define interfaces without classes

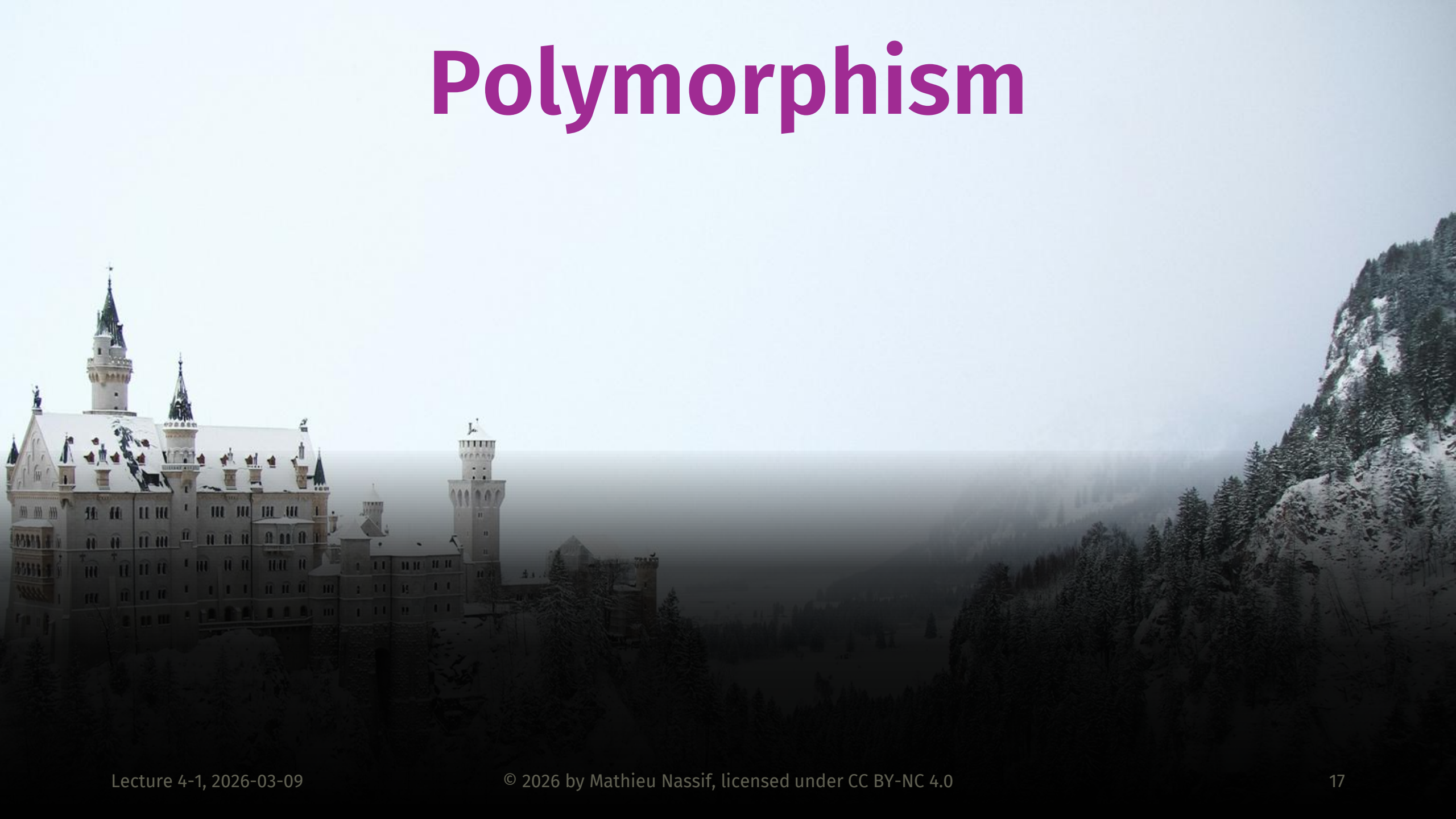
```
Link link = new Link("https://www.asciiart.eu", "Essential");
ContentFragment copy = link;
ContentFragment fragment = new Link("https://www.asciiart.eu", "Essential");

// ContentFragment fragment = new ContentFragment();    // ERROR
// Link extraCopy = copy;                               // ERROR

link.setDescriptor("ASCII art snippets");
// copy.setDescriptor("ASCII art snippets");            // ERROR
```

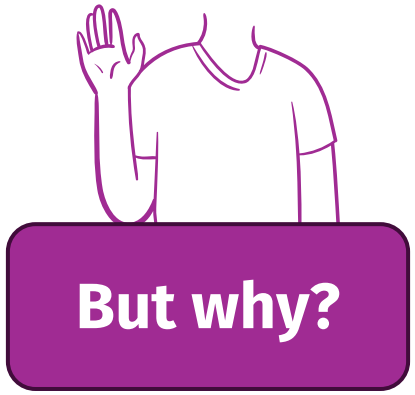

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Polymorphism




© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Define interfaces without classes




Interfaces can be reused for multiple classes

- `Link.showAsText()`
- `Note.toString()`
- `AsciiImage.getPrintableImage()`


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# The power of interfaces

```
public enum AsciiImage implements ContentFragment {
```

```
@Override
public String convertToString() {
    return ascii;
}
```

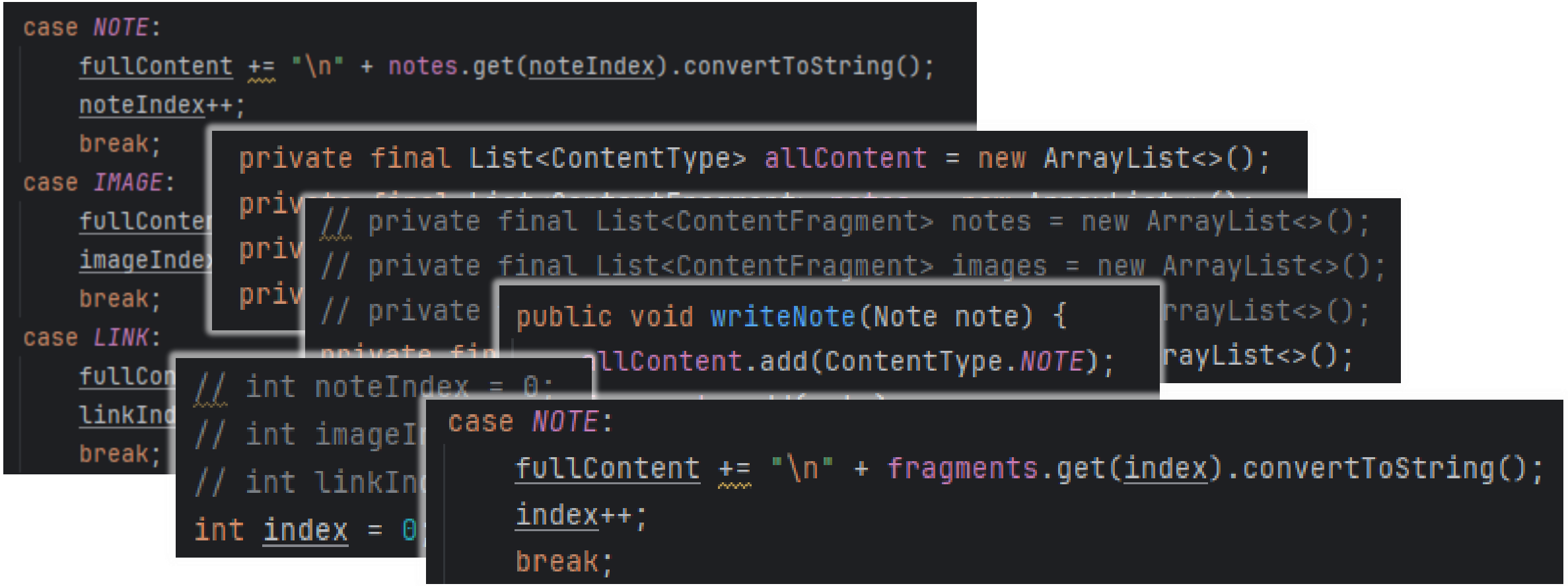
```
public record Note(String content) implements ContentFragment {
```

```
@Override
public String convertToString() {
    String[] lines = content.split("\n");

    String framedContent = "+----------------
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# The power of interfaces





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# The power of interfaces

```java
public String readAllNotes() {
    String fullContent = "Notebook for " + createCourseString();
    fullContent += "\n--------";
    for (ContentFragment fragment : fragments) {
        fullContent += "\n" + fragment.convertToString();
    }
    return fullContent;
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Polymorphism

*A variable can have many forms (types) that can be used **interchangeably***

```
for (ContentFragment fragment : fragments) {
    fullContent += "\n" + fragment.convertToString();
}
```

`Note`, `Link`, or `AsciiImage`

It doesn’t matter, all we rely on is the common interface.


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Summary

1. Creating a class creates both a blueprint and a type
2. We may want to reuse similar behaviours across classes
3. We use Java interfaces to define only a type (no blueprint)
4. Interfaces allow to use different types indiscriminately


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Classes can do more, but no less

- Interfaces are contracts about available methods


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# One more example: iterable Notebook

Any class that implements `Iterable` can be used in a for each loop

```
public interface Iterable<T> {
    Iterator<T> iterator();
}
```

```
public interface Iterator<E> {
    boolean hasNext();
    E next();
}
```

*Not the actual implementations


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# One more example: iterable Notebook

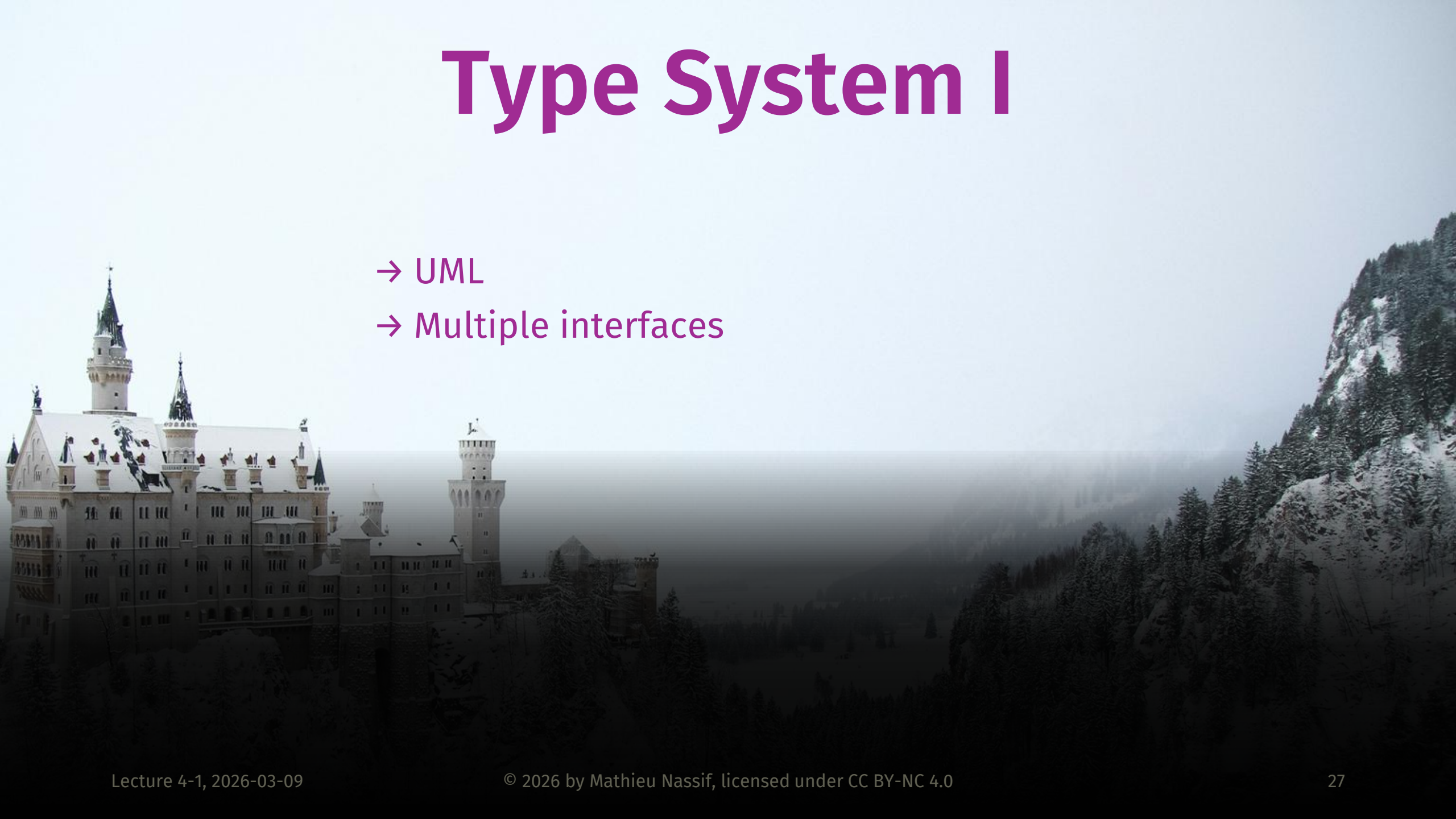
```
public class Notebook implements Iterable<ContentFragment> {
```

```
@Override
public Iterator<ContentFragment> iterator() {
    return fragments.iterator();
}
```

```
for (ContentFragment fragment : notebook) {
    System.out.println(fragment.convertToString());
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Type System I

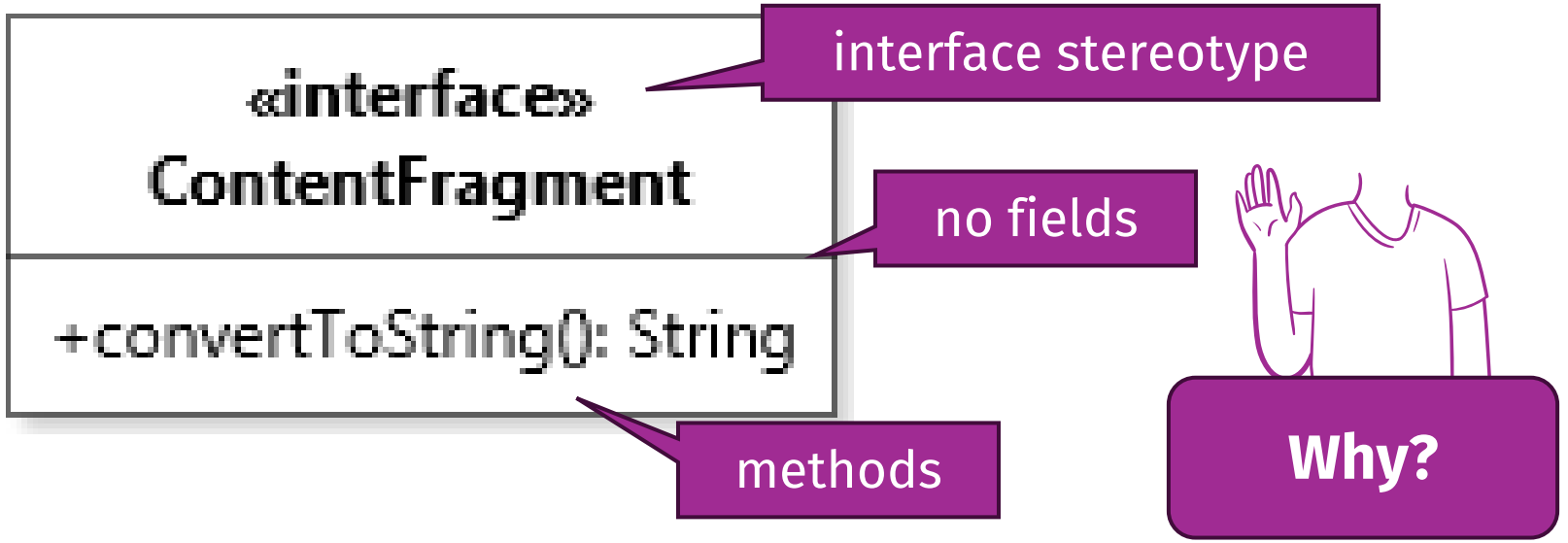


→ UML

→ Multiple interfaces


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

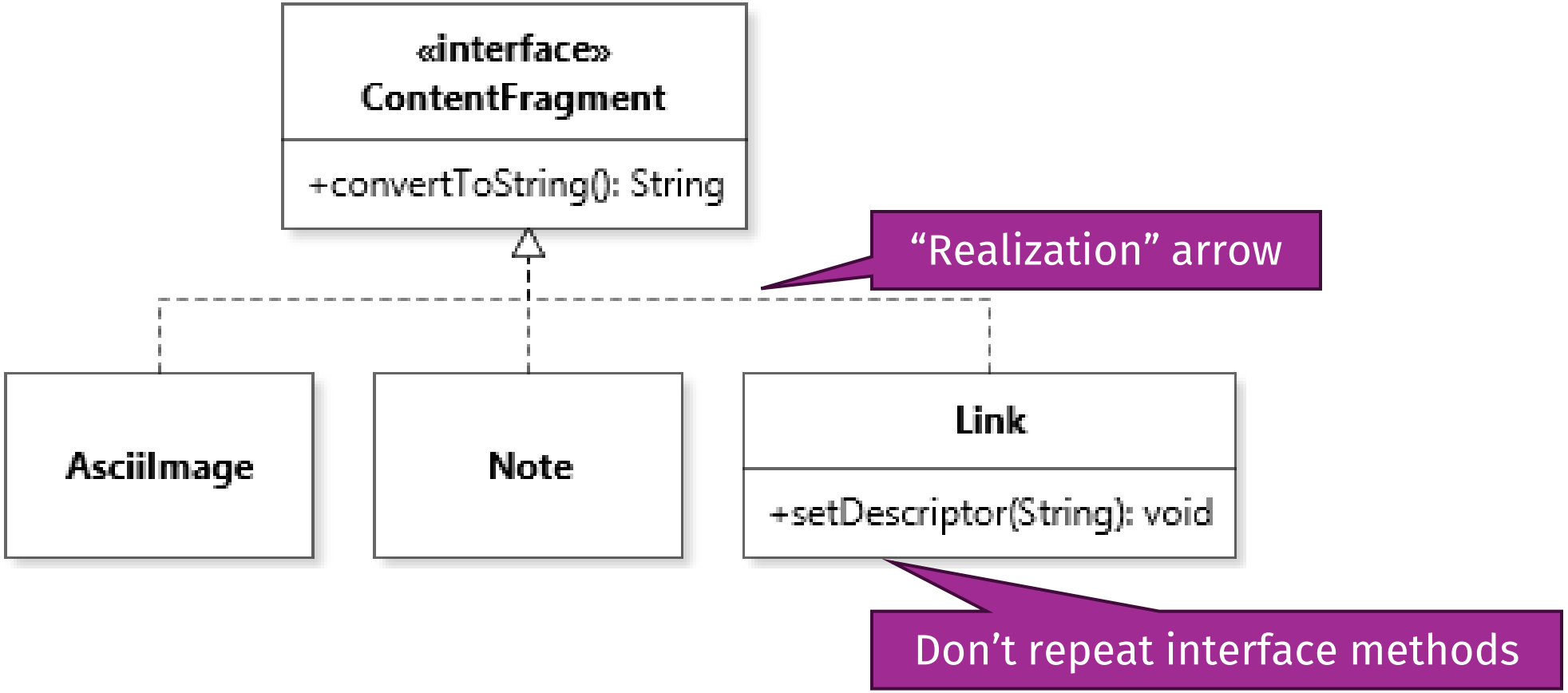
# Interfaces in UML class diagrams





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

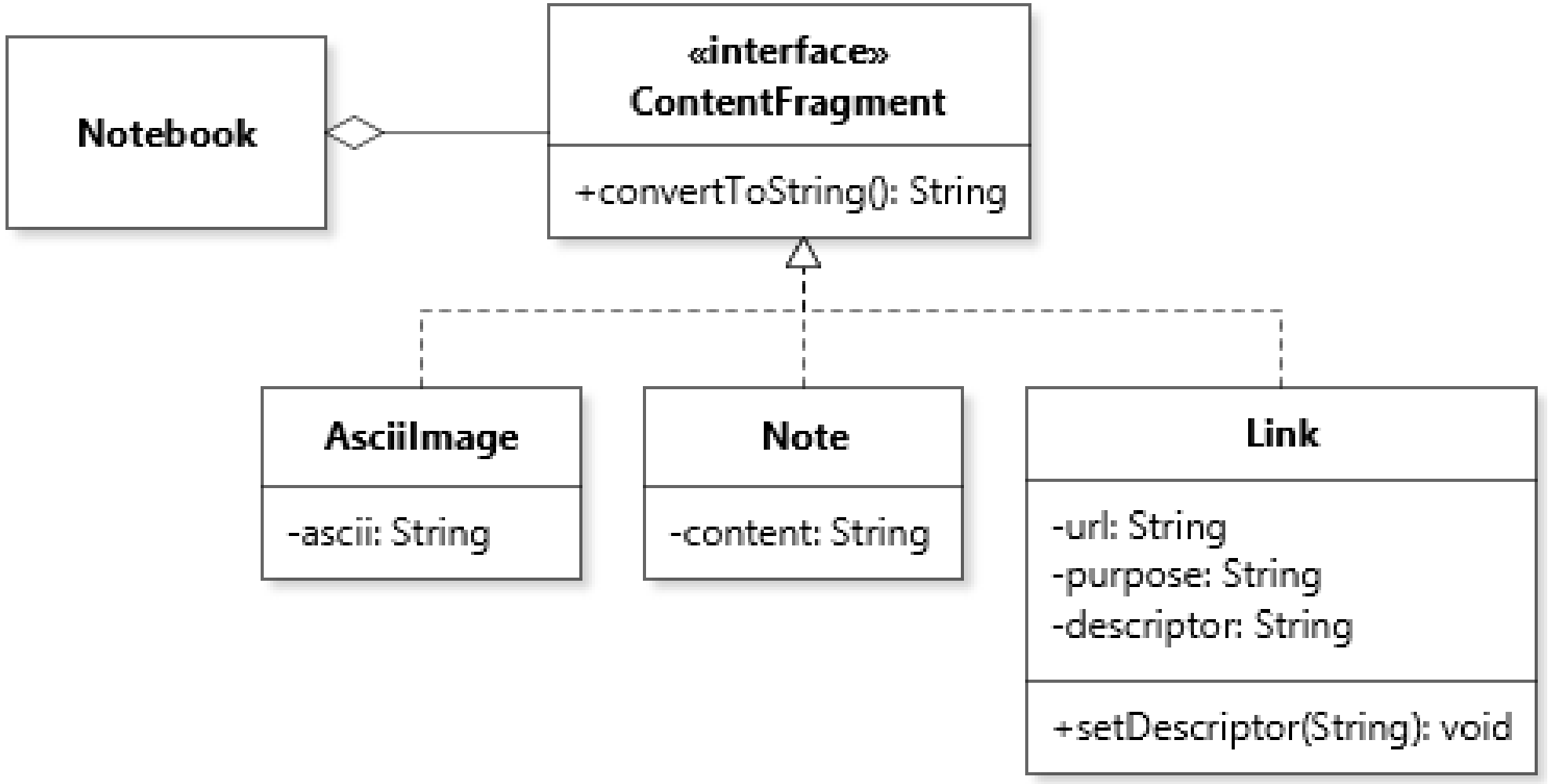
# Interfaces in UML class diagrams





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

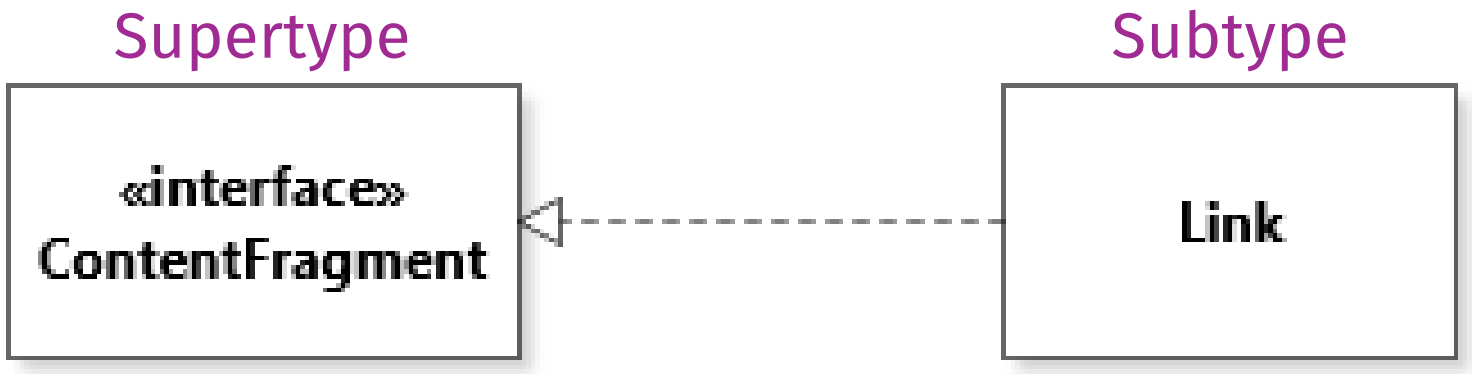
# Interfaces in UML class diagrams





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Types and subtypes




- Declaring an interfaces creates a type
- Declaring a class creates a type
- Implementing an interface creates a subtype


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Conversions

We can use assign values of a subtype
to variables of a supertype

```java
int small = 0;
long big = small;
```

```java
Link link = new Link("https://www.asciiart.eu", "Essential");
ContentFragment copy = link;
```





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Multiple interfaces

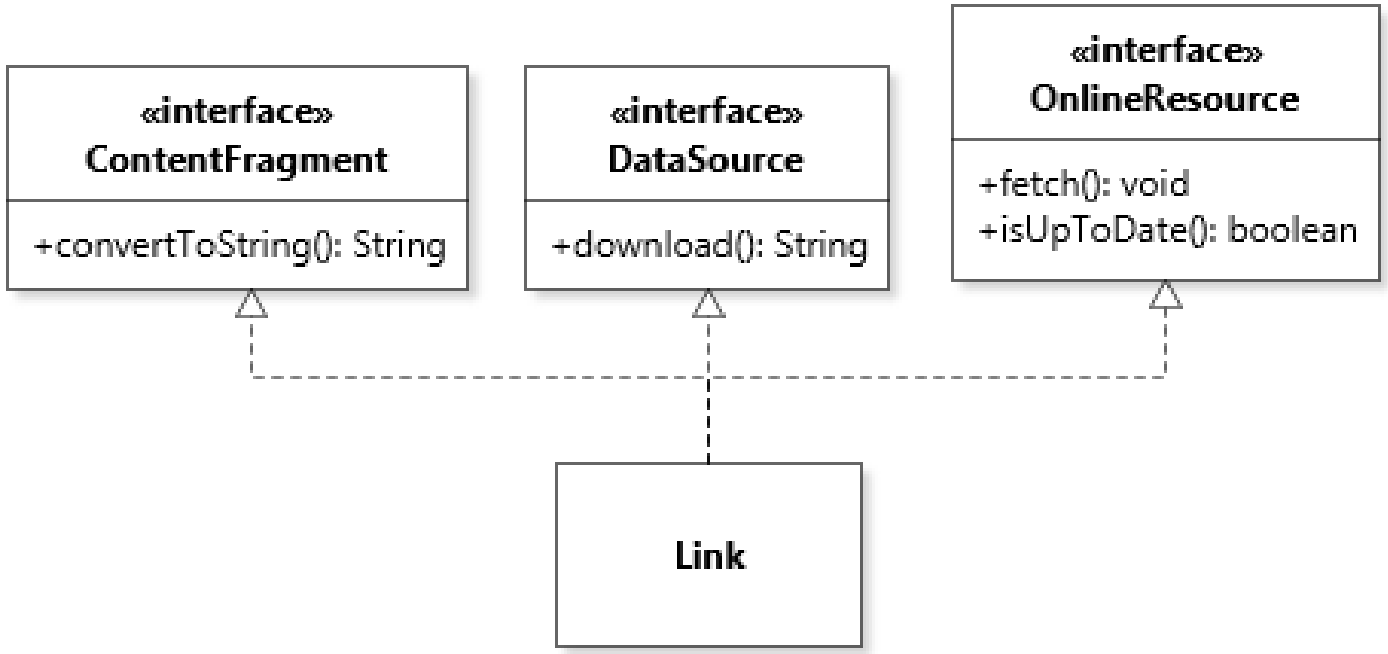
*A class can implement multiple interfaces*

```
public class Link implements ContentFragment, DataSource, OnlineResource {
```

```
@Override
public void fetch() {...}

@Override
public boolean isUpToDate() {...}

@Override
public String download() {...}
```

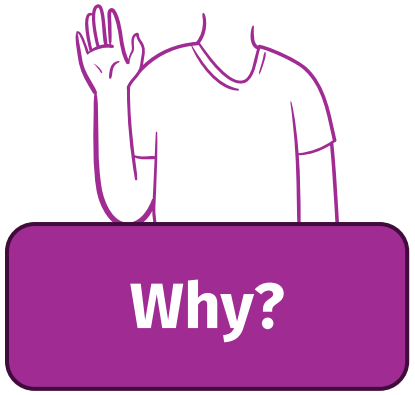


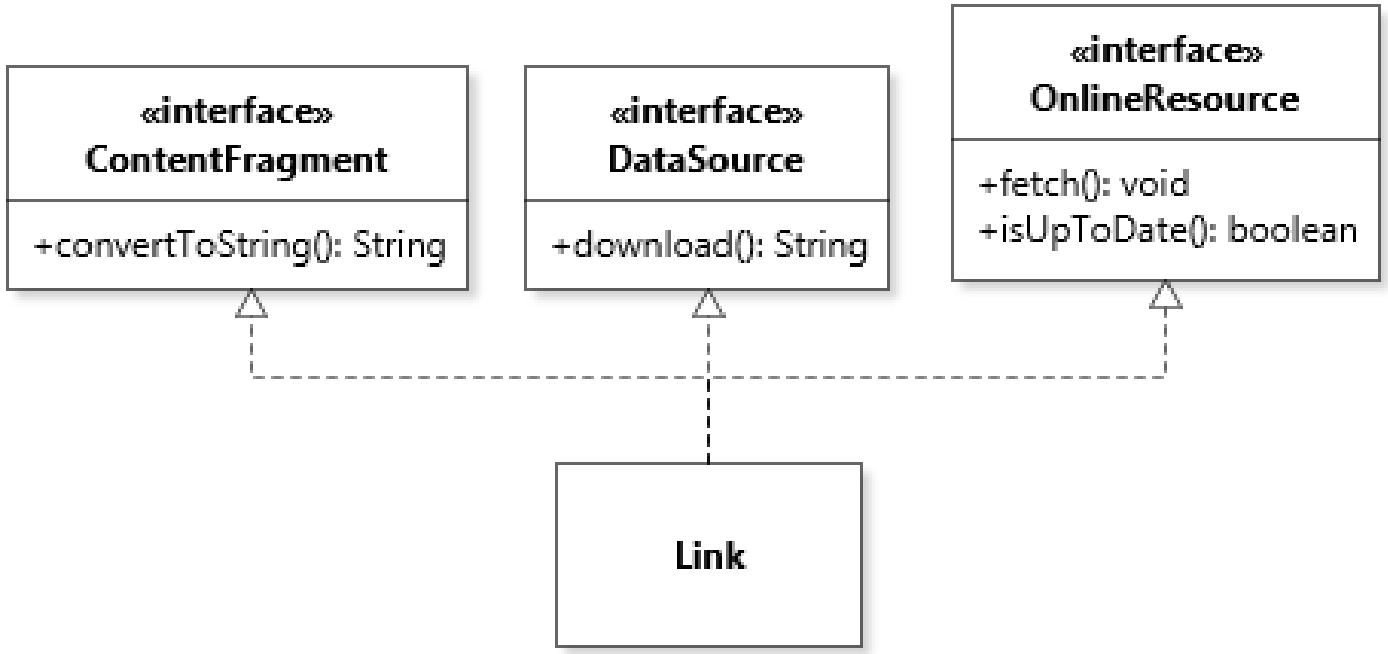


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Multiple interfaces

*A class can implement multiple interfaces*




- Keep interfaces small and focused


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

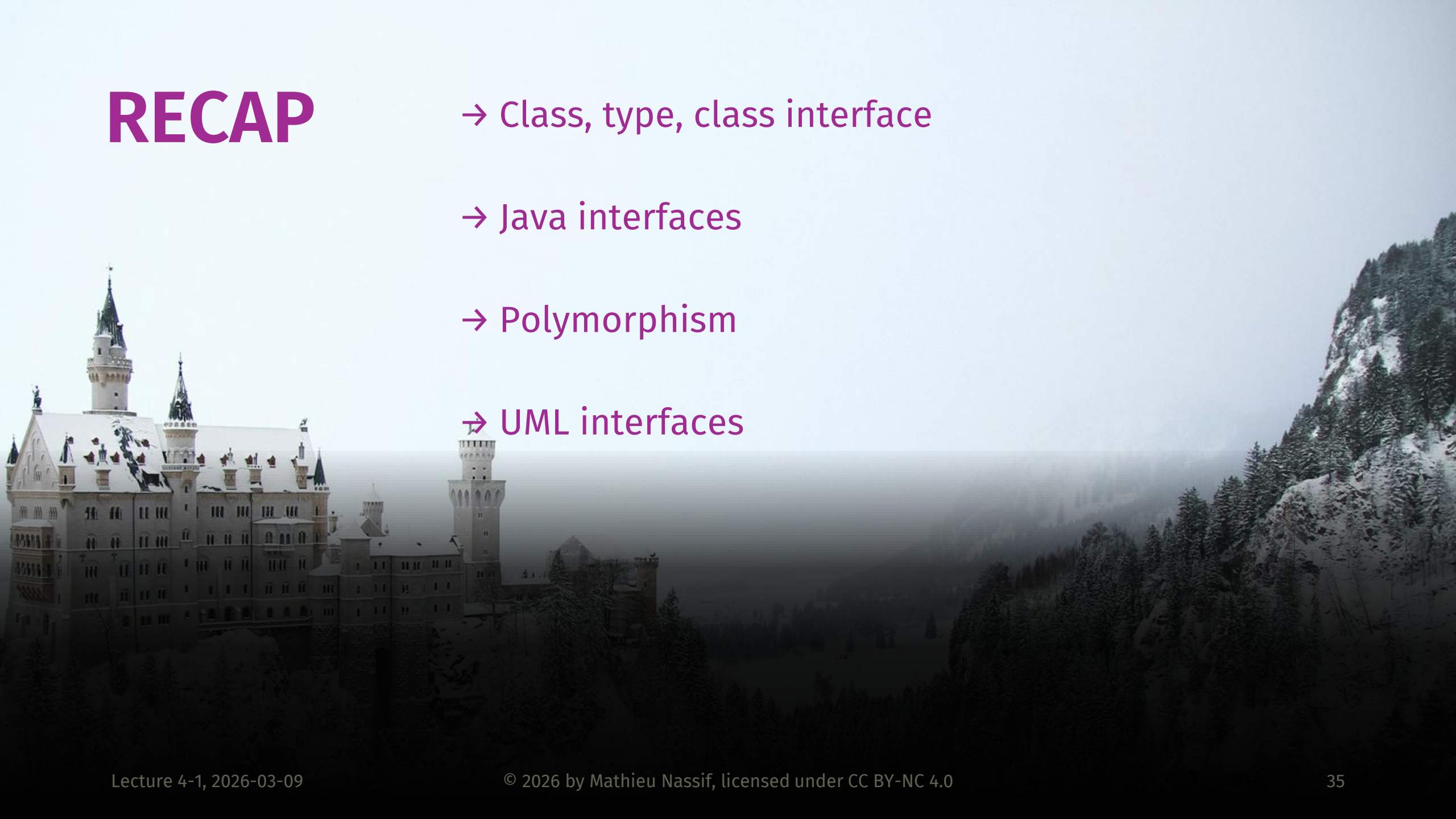
# RECAP

→ Class, type, class interface

→ Java interfaces

→ Polymorphism

→ UML interfaces




© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0